Corrigiendo inversiones, muchas a la vez

Adaptación a C#

Dos algoritmos, ambos $O(n \log n)$, ambos resolviendo el mismo problema desde extremos opuestos. El *merge sort* ordena cada mitad recursivamente primero, y luego entrelaza las dos mitades ordenadas. El *quicksort* particiona el arreglo alrededor de un pivote primero, y luego ordena cada lado recursivamente. Estas dos estrategias no son invenciones independientes. Son *duales* — dos formas de atacar el mismo problema estructural en órdenes opuestos.

La lente que hace visible la dualidad es la distinción entre inversiones *locales* y *globales*. Una vez que tienes la lente, los dos algoritmos dejan de parecer invenciones rivales y empiezan a parecer las dos estrategias naturales — arreglar las locales primero, o arreglar las globales primero — para cualquier ordenamiento de divide y vencerás.

Cada paso de merge y cada paso de partición arregla un número de inversiones proporcional a cuánto trabajo hizo el paso. Esa linealidad es el cambio de una inversión por operación a tantas inversiones como la operación pueda ver. La primera te costó $O(n^2)$ en el capítulo anterior; la segunda te compra $O(n \log n)$ aquí. El factor de $\log n$ en lugar de n entre los dos regímenes es lo que compra divide y vencerás.

1. Dos tipos de inversiones, dos órdenes de ataque

Toma cualquier arreglo y pártelo por la mitad en algún punto medio m. Toda inversión en el arreglo — todo par (i, j) con $i < j$ y `items`$[i] >$ `items`$[j]$ — cae en exactamente uno de tres cubos:

- **Local a la mitad izquierda:** tanto i como j están en `items[0..m]`. Ambos elementos viven del mismo lado del corte.
- **Local a la mitad derecha:** tanto i como j están en `items[m..n]`. La misma idea del otro lado.
- **Global (o cruzada):** i está en la mitad izquierda, j en la derecha. El par cruza el corte.

Suma los tres conteos y obtienes el conteo total de inversiones del arreglo. Para ordenar el arreglo, tienes que llevar los tres cubos a cero.

Aquí está el hecho: *cualquier* ordenamiento de divide y vencerás que parte el arreglo en dos mitades tiene que arreglar ambos tipos de inversión — local y global — y la elección algorítmica es cuál arreglar primero.

- **Locales primero.** Ordena recursivamente cada mitad, lo que arregla toda inversión local (ambas mitades terminan ordenadas, con cero inversiones internas). Lo que queda son solo inversiones cruzadas. Luego manéjalas en un barrido lineal que combina las dos mitades ordenadas. Esto es **merge sort**.
- **Globales primero.** Reorganiza el arreglo para que todo elemento de la mitad izquierda sea menor que todo elemento de la mitad derecha. Eso arregla toda inversión cruzada de un golpe (ahora hay cero pares que cruzan el corte y están fuera de orden). Lo que queda son solo inversiones locales dentro de cada lado, que las llamadas recursivas manejan. Esto es **quicksort**.

Los dos algoritmos no son invenciones rivales. Son los dos órdenes naturales en que divide y vencerás puede resolver el presupuesto de inversiones. Cualquiera de los dos órdenes paga el mismo precio: $O(n)$ trabajo por nivel de recursión, $O(\log n)$ niveles, total $O(n \log n)$. El trabajo por nivel es lineal porque cada paso de merge o partición es lineal; la profundidad es logarítmica porque cada corte parte la entrada a la mitad. El producto es el capítulo.

2. Merge sort: locales primero, luego entrelazar

Empieza con las locales. Ordena recursivamente la mitad izquierda y la mitad derecha. Después de que las dos llamadas recursivas regresen, ambas mitades están ordenadas, lo que significa que toda inversión local ha sido resuelta. Las únicas inversiones que aún pueden existir son las cruzadas — pares que cruzan el punto medio.

Ahora el paso de conquista. Dos secuencias ordenadas pueden combinarse en una sola secuencia ordenada en tiempo lineal: mantén un dedo en el frente de cada una, compara repetidamente los dos dedos, elige el menor, y avanza el dedo que ganó. Tras a lo sumo n comparaciones has consumido ambas listas.

```
public static void MergeSort<T>(
    IList<T> items, Comparison<T> order = null)
{
    order ??= Comparer<T>.Default.Compare;
    MergeSortRange(items, 0, items.Count, order);
}

private static void MergeSortRange<T>(
    IList<T> items, int lo, int hi, Comparison<T> order)
{
    if (hi - lo <= 1) return;
    int mid = lo + (hi - lo) / 2;
    var left = new T[mid - lo];
    var right = new T[hi - mid];
    for (int k = 0; k < left.Length; k++) left[k] = items[lo + k];
    for (int k = 0; k < right.Length; k++) right[k] = items[mid + k];

    MergeSortRange(items, lo, mid, order);
    MergeSortRange(items, mid, hi, order);
    Merge(items, lo, left, right, order);
}
```

Listing 1: Merge sort

La recursión parte en el punto medio, asigna dos listas nuevas para las mitades (porque las llamadas recursivas, de lo contrario, se sobrescribirían mutuamente el trabajo), ordena cada mitad, y luego las fusiona de vuelta en el almacenamiento original. El helper `Merge` es donde mueren las inversiones cruzadas.

```
private static void Merge<T>(
    IList<T> items, int offset,
    T[] left, T[] right, Comparison<T> order)
{
    int i = 0, j = 0, k = offset;
    while (i < left.Length && j < right.Length)
    {
        if (order(left[i], right[j]) <= 0)
        {
            items[k] = left[i];
            i++;
```

```
        }
        else
        {
            items[k] = right[j];
            j++;
        }
        k++;
    }
    while (i < left.Length) { items[k] = left[i]; i++; k++; }
    while (j < right.Length) { items[k] = right[j]; j++; k++; }
}
```

Listing 2: `Merge`: entrelaza dos mitades ordenadas

Tres bucles. El primero recorre ambas mitades en paralelo, eligiendo el elemento frontal menor cada vez. El segundo y tercero manejan lo que sobre una vez que un lado se agota. Cada iteración del primer bucle cuesta una comparación y escribe un elemento; los bucles de limpieza son copias puras. Trabajo total: lineal en la longitud combinada.

```
var items = new[] { 3, 1, 4, 1, 5, 9, 2, 6 };
MergeSortImpl.MergeSort(items);
Console.WriteLine(string.Join(", ", items)); // 1, 1, 2, 3, 4, 5, 6, 9

// Merge sort es estable con claves iguales: el <= en la comparacion
// elige el izquierdo primero cuando hay empate.
var pairs = new[] { (1, 'a'), (2, 'b'), (1, 'c'), (2, 'd') };
Comparison<(int, char)> byFirst = (x, y) => x.Item1.CompareTo(y.Item1);
MergeSortImpl.MergeSort(pairs, byFirst);
Console.WriteLine(string.Join(", ", pairs)); // (1,a), (1,c), (2,b), (2,
    d)
```

```
1, 1, 2, 3, 4, 5, 6, 9
(1, a), (1, c), (2, b), (2, d)
```

Ahora la historia de las inversiones. Mira la rama `else` en `Merge` — la rama que toma del lado derecho. Cuando se toma `right[j]` antes que `left[i]`, eso significa `right[j]` $<$ `left[i]`. Pero `left` ya está ordenado (la llamada recursiva se encargó de eso), así que `left[i]` $\leq$ `left[i+1]` $\leq \cdots \leq$ `left[left.Length-1]`. Lo que significa que `right[j]` es menor que todo elemento restante en la mitad izquierda.

Eso no es *una* inversión cruzada arreglada. Eso es `left.Length` $- i$ inversiones cruzadas arregladas, en una sola operación. El paso de merge procesa cada inversión cruzada en el momento en que toma del lado derecho, y hay a lo sumo $O(n)$ tales tomas para un arreglo de tamaño n. El merge entero es lineal, y resuelve toda inversión cruzada en una pasada.

Esa es la estrategia de locales primero en acción. Las llamadas recursivas eliminaron todas las inversiones locales antes de que el merge corriera. El merge toma una entrada izquierda-ordenada + derecha-ordenada — un arreglo cuyas únicas inversiones restantes son cruzadas — y las mata en $O(n)$. Trabajo lineal, todas las inversiones restantes arregladas. Esa es la fuente de la mejora de $\log n$ sobre el ordenamiento básico.

3. Contar inversiones, gratis, en $O(n \log n)$

El `CountInversions` de fuerza bruta del capítulo 3 recorría todos los pares en $O(n^2)$ — bien como ayuda didáctica, inútil a escala. Merge sort te da un algoritmo mucho mejor casi gratis.

La idea es la de la sección anterior. Cada vez que `Merge` toma del lado derecho, ha arreglado `left.Length` $- i$ inversiones cruzadas. Si instrumentas el merge para *contar* esas correcciones

en lugar de solo ejecutarlas, has contado toda inversión global en el arreglo. Recurre sobre las mitades para contar sus inversiones internas (locales) de la misma forma, y has contado el total: local-izquierda + local-derecha + global.

```
public static int CountInversionsFast<T>(
    IReadOnlyList<T> items, Comparison<T> order = null)
{
    order ??= Comparer<T>.Default.Compare;
    var work = new T[items.Count];
    for (int k = 0; k < items.Count; k++) work[k] = items[k];
    return MergeSortCounting(work, 0, work.Length, order);
}

private static int MergeSortCounting<T>(
    T[] items, int lo, int hi, Comparison<T> order)
{
    if (hi - lo <= 1) return 0;
    int mid = lo + (hi - lo) / 2;
    var left = new T[mid - lo];
    var right = new T[hi - mid];
    for (int k = 0; k < left.Length; k++) left[k] = items[lo + k];
    for (int k = 0; k < right.Length; k++) right[k] = items[mid + k];

    int count = MergeSortCounting(items, lo, mid, order);
    count += MergeSortCounting(items, mid, hi, order);
    count += MergeCounting(items, lo, left, right, order);
    return count;
}

private static int MergeCounting<T>(
    T[] items, int offset,
    T[] left, T[] right, Comparison<T> order)
{
    int count = 0;
    int i = 0, j = 0, k = offset;
    while (i < left.Length && j < right.Length)
    {
        if (order(left[i], right[j]) <= 0)
        {
            items[k] = left[i];
            i++;
        }
        else
        {
            items[k] = right[j];
            j++;
            count += left.Length - i; // inversiones cruzadas resueltas
                por esta toma
        }
        k++;
    }
    while (i < left.Length) { items[k] = left[i]; i++; k++; }
    while (j < right.Length) { items[k] = right[j]; j++; k++; }
    return count;
}
```

Listing 3: Conteo de inversiones en $O(n \log n)$ vía merge sort

La estructura es idéntica a merge sort. La única adición es la línea `count += left.Length`

- `i` dentro de la rama `else` — la única línea que registra "esta toma de la derecha acaba de resolver esta cantidad de inversiones cruzadas". Las llamadas recursivas manejan las inversiones locales dentro de cada mitad contando sus propias inversiones cruzadas un nivel más abajo. En cada nivel de la recursión, contabilizas las globales de ese nivel; suma a través de los niveles y tienes el total.

```
var items = new[] { 3, 1, 4, 1, 5, 9, 2, 6 };
Console.WriteLine(Basic.CountInversions(items));                // 11
Console.WriteLine(MergeSortImpl.CountInversionsFast(items));   // 11
```

```
11
11
```

La misma forma algorítmica, la misma comparación única por paso de merge — y obtuviste una respuesta extra gratis. Este es un patrón recurrente en el diseño de algoritmos: cuando un procedimiento ya tiene una propiedad estructural útil (aquí, "cada toma de la derecha resuelve un número conocido de inversiones cruzadas"), pequeñas modificaciones pueden extraer respuestas adicionales sin cambiar el coste asintótico.

4. Quicksort: globales primero, luego ordenar cada lado

La misma meta que merge sort, orden de ataque opuesto. En lugar de arreglar las inversiones locales primero vía recursión y luego entrelazar las mitades, quicksort arregla las inversiones globales primero reorganizando el arreglo, luego maneja las locales vía recursión.

La reorganización es el *paso de partición*. Elige un elemento pivote (usaré el último elemento del intervalo), y luego recorre el resto e intercambia cosas de modo que todo lo menor que el pivote aterrice a la izquierda del pivote, y todo lo no menor aterrice a su derecha.

```
/// <summary>
/// Particion de Lomuto sobre items[lo..hi). Trata items[hi - 1]
/// como el pivote. Reorganiza in situ para que los elementos
/// menores que el pivote queden a su izquierda, y los no menores
/// a su derecha. Devuelve el indice final del pivote.
/// </summary>
public static int Partition<T>(
    IList<T> items, int lo, int hi, Comparison<T> order)
{
    T pivot = items[hi - 1];
    int i = lo;
    for (int j = lo; j < hi - 1; j++)
    {
        if (order(items[j], pivot) < 0)
        {
            (items[i], items[j]) = (items[j], items[i]);
            i++;
        }
    }
    (items[i], items[hi - 1]) = (items[hi - 1], items[i]);
    return i;
}
```

Listing 4: Partición de Lomuto

Dos índices hacen el trabajo. j recorre el intervalo buscando elementos menores que el pivote. i marca el borde derecho de la región "menor" — todo elemento a la izquierda de i ya se sabe menor que el pivote. Cada vez que j encuentra otro elemento menor, lo intercambia a la posición

i y avanza i. Cuando el escaneo termina, intercambia el pivote mismo a la posición i, donde pertenece.

Aquí está la historia de las inversiones globales. Después de que la partición devuelve el índice final del pivote p, todo elemento en `items[lo..p]` es estrictamente menor que `items[p]`, y todo elemento en `items[p+1..hi]` es al menos `items[p]`. Lo que significa: toda inversión cruzada entre la futura mitad izquierda (elementos debajo de p) y la futura mitad derecha (elementos en p o por encima) se ha ido. Ningún par de elementos que cruza p está fuera de orden. El conteo de inversiones globales, relativo al corte en p, ahora es cero.

Eso es una gran cantidad de inversiones limpiadas por una sola pasada lineal. La partición hizo $O(n)$ comparaciones y un número pequeño de intercambios; a cambio, arregló toda inversión cruzada que involucra el "lado" del pivote. Lo que queda son las inversiones dentro de cada lado — las locales — y las llamadas recursivas se encargan de ellas.

```
public static void QuickSort<T>(
    IList<T> items, Comparison<T> order = null)
{
    order ??= Comparer<T>.Default.Compare;
    QuickSortRange(items, 0, items.Count, order);
}

private static void QuickSortRange<T>(
    IList<T> items, int lo, int hi, Comparison<T> order)
{
    if (hi - lo < 2) return;
    int pivotIndex = Partition(items, lo, hi, order);
    QuickSortRange(items, lo, pivotIndex, order);
    QuickSortRange(items, pivotIndex + 1, hi, order);
}
```

Listing 5: Quicksort

El `QuickSort` público es solo una envoltura delgada que inicializa los límites de la recursión. El helper `QuickSortRange` toma `lo` y `hi` explícitos para que la recursión pueda trabajar *in situ* sobre subrangos sin copiar. La estructura refleja perfectamente a merge sort, pero léela al revés: la partición (el paso que mata globales) viene *antes* de las llamadas recursivas (los pasos que matan locales), no después.

```
var items = new[] { 3, 1, 4, 1, 5, 9, 2, 6 };
QuickSortImpl.QuickSort(items);
Console.WriteLine(string.Join(", ", items)); // 1, 1, 2, 3, 4, 5, 6, 9

// Partition esta expuesta porque el siguiente capitulo la reutiliza
// para quickselect.
var items2 = new[] { 3, 1, 4, 1, 5, 9, 2, 6 };
int p = QuickSortImpl.Partition(items2, 0, items2.Length,
                                (a, b) => a.CompareTo(b));
Console.WriteLine($"despues de particionar: [{string.Join(", ", items2)
    }], " +
                  $"pivote en indice {p}, valor {items2[p]}");
// Todo elemento antes de p es < items2[p]; todo elemento
// despues es >= items2[p]. Cero inversiones cruzadas entre
// los dos lados: las globales se fueron en una sola pasada.
```

```
1, 1, 2, 3, 4, 5, 6, 9
despues de particionar: [3, 1, 4, 1, 5, 2, 6, 9], pivote en indice 6, valor 6
```

Una cosa para señalar, porque importará en el análisis. La versión de quicksort de arriba siempre elige el último elemento del intervalo como pivote. Esa es la elección de pivote más

simple posible, y sobre entradas aleatorias funciona de maravilla. Pero sobre ciertas entradas patológicas — ya ordenadas, ordenadas al revés, o todas iguales — es catastróficamente mala. Elegir el último elemento de un intervalo ordenado es la peor elección posible: la partición parte el arreglo en una pieza de tamaño $n - 1$ y una pieza de tamaño cero, la recursión se vuelve efectivamente lineal, y quicksort degenera a $O(n^2)$.

El arreglo estándar es **selección aleatoria del pivote** — elegir un pivote uniformemente al azar del intervalo, luego intercambiarlo a la última posición antes de particionar. Esto hace que el comportamiento del peor caso requiera una entrada del peor caso y una secuencia del peor caso de elecciones aleatorias, que juntas ocurren con probabilidad que se desvanece. El siguiente capítulo (sobre selección) muestra la versión aleatorizada completa; la versión de aquí es la exposición más clara de la idea de partición, y la mantengo así a propósito.

5. Las tres preguntas, aplicadas

¿Es correcto?

Ambos algoritmos tienen argumentos de corrección recursiva limpios construidos sobre la descomposición local/global. Para merge sort, el caso inductivo es: asume que las llamadas recursivas producen mitades ordenadas (cero inversiones locales en cada una), entonces `Merge` correctamente las entrelaza en un todo ordenado porque siempre elige el menor de los dos frentes — así que cualquier inversión cruzada restante tendría que poner el elemento mayor primero, lo cual el algoritmo se niega a hacer.

Para quicksort, el invariante de `Partition` hace el trabajo: después de que la partición devuelve el índice p, todo elemento en `items[lo..p]` es menor que `items[p]`, y todo elemento en `items[p+1..hi]` no es menor. Así que `items[p]` está en su posición final ordenada, y las dos llamadas recursivas manejan los dos lados — que solo pueden contener inversiones locales — independientemente.

En ambos casos, el caso base (una secuencia de longitud cero o uno) no tiene inversiones de ningún tipo.

¿Cuán eficiente es?

Merge sort es tiempo $O(n \log n)$ y espacio extra $O(n)$, sobre toda entrada. La recursión tiene profundidad $\log_2 n$ (cada nivel parte el intervalo a la mitad), y cada nivel hace $O(n)$ trabajo total a través de todos los merges de ese nivel. Combina y obtienes $n \log n$. El coste de espacio viene de las listas auxiliares `left` y `right` asignadas en cada llamada recursiva — en cualquier momento hasta $O(n)$ de almacenamiento extra está vivo.

Quicksort es tiempo $O(n \log n)$ en promedio y espacio extra $O(\log n)$ en promedio (solo la pila de recursión). La profundidad de la recursión depende de la calidad del pivote: un pivote balanceado (cercano a la mediana) da profundidad $\log_2 n$; un pivote desbalanceado da más. Sobre una entrada uniformemente aleatoria con pivote del último elemento, la profundidad promedio es $O(\log n)$ y el trabajo total promedio es $O(n \log n)$. Sobre las entradas patológicas señaladas arriba, la profundidad se vuelve $O(n)$ y el trabajo total se vuelve $O(n^2)$. La selección aleatoria del pivote hace que el mal caso sea una curiosidad probabilística en lugar de un riesgo estructural.

En el caso promedio los dos empatan. Divergen en estabilidad (merge sort preserva el orden relativo de claves iguales; quicksort no, al menos en esta implementación) y en memoria (merge sort necesita $O(n)$ espacio de trabajo; quicksort se las arregla con $O(\log n)$ marcos de pila).

¿Es óptimo?

Este es el golpe del capítulo, y es grande. La cota inferior del ordenamiento basado en comparaciones es $\Omega(n \log n)$. Ningún algoritmo que aprenda sobre su entrada solo comparando pares

de elementos puede ordenar una entrada arbitraria en menos de $\Omega(n \log n)$ comparaciones en el peor caso. Merge sort cumple esta cota exactamente; quicksort la cumple en promedio. Dentro del modelo de comparaciones, no puedes hacerlo asintóticamente mejor que lo que este capítulo acaba de mostrarte.

El argumento es la cota inferior del árbol de decisión para ordenamiento, la misma forma que dio la cota $\Omega(\log n)$ para búsqueda en el capítulo anterior, escalada. Cualquier ordenamiento basado en comparaciones puede dibujarse como un árbol de decisión binario: cada nodo interno es una comparación ("¿es `items[i]` < `items[j]`?"), cada hoja es una permutación de salida específica. Para manejar toda entrada posible correctamente, el árbol necesita al menos $n!$ hojas — una por cada ordenamiento posible de la entrada. Un árbol binario con L hojas tiene profundidad al menos $\log_2 L$, así que la profundidad es al menos $\log_2(n!)$. Por la aproximación de Stirling — el hecho de que $n!$ crece como $(n/e)^n$, así que $\log_2(n!) \approx n \log_2 n$ — esa profundidad es $\Theta(n \log n)$. El camino del peor caso a través de cualquier árbol así es $\Omega(n \log n)$ comparaciones.

Teorema 1. *Cualquier algoritmo de ordenamiento basado en comparaciones requiere $\Omega(n \log n)$ comparaciones en el peor caso. Merge sort cumple esta cota exactamente; quicksort la cumple en promedio.*

Esa es una afirmación fuerte. Dice: el ordenamiento basado en comparaciones nunca puede ser lineal, no importa cuán astuto sea el algoritmo. El factor de $\log n$ sobre lo lineal es el precio que pagas por no saber nada sobre la entrada excepto cómo se comparan sus elementos. Los siguientes dos capítulos exploran ambos lados de ese precio: *selección* (capítulo 5) muestra que puedes obtener información parcialmente ordenada más rápido que el ordenamiento completo, y *ordenamiento en tiempo lineal* (capítulo 6) muestra que puedes romper la cota inferior por completo si abandonas la restricción de solo comparaciones.

6. Dos órdenes, un coste

Merge sort y quicksort son correctos vía argumentos recursivos limpios — mitades ordenadas se combinan en un todo ordenado; la partición coloca el pivote en su posición final. Ambos corren en $O(n \log n)$, merge sort siempre y quicksort en promedio. Y ambos son óptimos: la cota inferior del árbol de decisión dice que ningún ordenamiento basado en comparaciones puede hacerlo mejor que $\Omega(n \log n)$ en el peor caso, y estos algoritmos cumplen esa cota.

La dualidad local-vs-global es el movimiento que generaliza. Cualquier ordenamiento de divide y vencerás que parte su entrada en dos mitades tiene que arreglar dos tipos de inversiones — locales dentro de cada mitad, globales que cruzan el corte — y la elección algorítmica es en qué orden arreglarlas. Merge sort arregla las locales primero vía recursión, luego mata las globales en un merge lineal. Quicksort mata las globales primero vía partición, luego arregla las locales vía recursión. Cualquiera de los dos órdenes funciona. Ambos pagan el mismo factor $\log n$ por profundidad de recursión, y ambos se benefician del mismo truco de muchas-a-la-vez en cada nivel. Los dos algoritmos son reflejos el uno del otro a través de la división local/global.

El mismo argumento de conteo de inversiones del capítulo 3 generaliza. Los ordenamientos básicos arreglan una inversión por intercambio, así que están atascados en $O(n^2)$ porque puede haber esa cantidad de inversiones. Los ordenamientos eficientes arreglan inversiones en bloques proporcionales al trabajo hecho en cada nivel — un paso de merge hace $O(n)$ trabajo y resuelve $O(n)$ inversiones cruzadas; un paso de partición hace $O(n)$ trabajo y resuelve todas las inversiones cruzadas del pivote en un solo intercambio. El truco que convierte n^2 en $n \log n$ es exactamente el truco de acreditarte por todas las inversiones que puedes ver en este nivel, en lugar de solo el par que casualmente intercambiaste.

En el siguiente capítulo, la pregunta cambia: ¿y si no necesitas el arreglo entero ordenado, sino solo el k-ésimo elemento más pequeño? Quicksort ya coloca un elemento por paso de partición en su posición final. La selección resulta requerir solo una llamada recursiva por nivel — no dos — y el coste colapsa de $O(n \log n)$ a $O(n)$.

7. Código completo

```
using System;
using System.Collections.Generic;

namespace Codex.Sort
{
    public static class MergeSortImpl
    {
        // ---------- Merge sort ----------
        public static void MergeSort<T>(
            IList<T> items, Comparison<T> order = null)
        {
            order ??= Comparer<T>.Default.Compare;
            MergeSortRange(items, 0, items.Count, order);
        }

        private static void MergeSortRange<T>(
            IList<T> items, int lo, int hi, Comparison<T> order)
        {
            if (hi - lo <= 1) return;
            int mid = lo + (hi - lo) / 2;
            var left = new T[mid - lo];
            var right = new T[hi - mid];
            for (int k = 0; k < left.Length; k++) left[k] = items[lo + k
                ];
            for (int k = 0; k < right.Length; k++) right[k] = items[mid
                + k];

            MergeSortRange(items, lo, mid, order);
            MergeSortRange(items, mid, hi, order);
            Merge(items, lo, left, right, order);
        }

        private static void Merge<T>(
            IList<T> items, int offset,
            T[] left, T[] right, Comparison<T> order)
        {
            int i = 0, j = 0, k = offset;
            while (i < left.Length && j < right.Length)
            {
                if (order(left[i], right[j]) <= 0)
                {
                    items[k] = left[i]; i++;
                }
                else
                {
                    items[k] = right[j]; j++;
                }
                k++;
            }
            while (i < left.Length) { items[k] = left[i]; i++; k++; }
            while (j < right.Length) { items[k] = right[j]; j++; k++; }
        }

        // ---------- Conteo de inversiones O(n log n) ----------
        public static int CountInversionsFast<T>(
            IReadOnlyList<T> items, Comparison<T> order = null)
```

```
        {
            order ??= Comparer<T>.Default.Compare;
            var work = new T[items.Count];
            for (int k = 0; k < items.Count; k++) work[k] = items[k];
            return MergeSortCounting(work, 0, work.Length, order);
        }

        private static int MergeSortCounting<T>(
            T[] items, int lo, int hi, Comparison<T> order)
        {
            if (hi - lo <= 1) return 0;
            int mid = lo + (hi - lo) / 2;
            var left = new T[mid - lo];
            var right = new T[hi - mid];
            for (int k = 0; k < left.Length; k++) left[k] = items[lo + k
                ];
            for (int k = 0; k < right.Length; k++) right[k] = items[mid
                + k];

            int count = MergeSortCounting(items, lo, mid, order);
            count += MergeSortCounting(items, mid, hi, order);
            count += MergeCounting(items, lo, left, right, order);
            return count;
        }

        private static int MergeCounting<T>(
            T[] items, int offset,
            T[] left, T[] right, Comparison<T> order)
        {
            int count = 0;
            int i = 0, j = 0, k = offset;
            while (i < left.Length && j < right.Length)
            {
                if (order(left[i], right[j]) <= 0)
                {
                    items[k] = left[i]; i++;
                }
                else
                {
                    items[k] = right[j]; j++;
                    count += left.Length - i;
                }
                k++;
            }
            while (i < left.Length) { items[k] = left[i]; i++; k++; }
            while (j < right.Length) { items[k] = right[j]; j++; k++; }
            return count;
        }
    }
}
```

Listing 6: `MergeSort.cs` — merge sort, conteo de inversiones

```
using System;
using System.Collections.Generic;

namespace Codex.Sort
{
    public static class QuickSortImpl
```

```
    {
        public static void QuickSort<T>(
            IList<T> items, Comparison<T> order = null)
        {
            order ??= Comparer<T>.Default.Compare;
            QuickSortRange(items, 0, items.Count, order);
        }

        private static void QuickSortRange<T>(
            IList<T> items, int lo, int hi, Comparison<T> order)
        {
            if (hi - lo < 2) return;
            int pivotIndex = Partition(items, lo, hi, order);
            QuickSortRange(items, lo, pivotIndex, order);
            QuickSortRange(items, pivotIndex + 1, hi, order);
        }

        // Particion de Lomuto sobre items[lo..hi). Pivote: items[hi -
            1].
        public static int Partition<T>(
            IList<T> items, int lo, int hi, Comparison<T> order)
        {
            T pivot = items[hi - 1];
            int i = lo;
            for (int j = lo; j < hi - 1; j++)
            {
                if (order(items[j], pivot) < 0)
                {
                    (items[i], items[j]) = (items[j], items[i]);
                    i++;
                }
            }
            (items[i], items[hi - 1]) = (items[hi - 1], items[i]);
            return i;
        }
    }
}
```

Listing 7: `QuickSort.cs` — quicksort y partición de Lomuto

8. Notas y lecturas adicionales

Merge sort se remonta a John von Neumann (1945) y se trata en CLRS (4.ª ed.) §2.3 (introducción) y §4.4–4.5 (análisis de recurrencias). Quicksort fue la invención de Tony Hoare en 1959 y está en CLRS §7; *Algorithms* (4.ª ed.) de Sedgewick §2.3 da el tratamiento de estilo producción más limpio con partición de tres vías y variantes de selección de pivote. La cota inferior del árbol de decisión está en CLRS §8.1 — la aproximación de Stirling es el golpe que convierte $\log_2(n!)$ en $n \log n$. La variante de conteo de inversiones en $O(n \log n)$ de merge sort es una observación folklore que aparece como ejercicio en casi todo libro de algoritmos; la derivación más limpia que conozco está en *Algorithm Design* de Kleinberg y Tardos §5.3. El artículo original de Hoare sobre Quicksort (1962) es corto, lúcido, y vale la pena leerlo como pieza de prosa algorítmica histórica. El enfoque de inversiones local-vs-global no es vocabulario estándar de libro de texto, pero es la lente organizadora más limpia que conozco para ver merge sort y quicksort como duales — ambos están presentes implícitamente en cualquier tratamiento completo del ordenamiento de divide y vencerás.